

CCS334 – BIG DATA ANALYTICS
LAB RECORD

Name :

Reg. No :

Branch :

Year/Sem :

**DEPARTMENT
OF
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

*Certified that this is the bonafide record of work done
by..... in the **CCS334 BIG DATA ANALYTICS** of
this institution, as prescribed by Anna University, Chennai for the year/semester
during the year 2022-2023.*

Staff-in-charge

Head of the Department

REGISTER NO.:

Submitted for the Year / Semester Examination of the Institution
conducted on

Internal Examiner

External Examiner

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE

INSTITUTION VISION & MISSION

VISION

To impart quality technical education emphasizing innovations and research with social and ethical values.

MISSION

- Establishing state-of-the-art infrastructure, effective procedures for recruitment of competent faculty and innovative teaching practices.
- Creating a conducive environment for nurturing innovative ideas and encouraging research skills.
- Inculcating social and ethical values through co-curricular and extra-curricular activities.

DEPARTMENT VISION & MISSION

VISION

To become centre of excellence in technical education to produce competent professionals to the dynamic needs in the field of Artificial Intelligence and Data Science.

MISSION

- Educating the fundamentals of AI&DS by continuously improving the teaching learning methodologies using modern tools.
- Creating a conducive environment for learning and research-based activities.
- Inculcating the interdisciplinary skill set required to fulfil the societal need.

COURSE OBJECTIVES:

- To understand big data.
- To learn and use NoSQL big data management.
- To learn mapreduce analytics using Hadoop and related tools.
- To work with map reduce applications

LIST OF EXPERIMENTS:

1. Downloading and installing Hadoop; Understanding different Hadoop modes. Startup scripts, Configuration files.
2. Hadoop Implementation of file management tasks, such as Adding files and directories, retrieving files and Deleting files
3. Implement of Matrix Multiplication with Hadoop Map Reduce
4. Run a basic Word Count Map Reduce program to understand Map Reduce Paradigm.
5. Installation of Hive along with practice examples.
6. Installation of HBase, Installing thrift along with Practice examples
7. Practice importing and exporting data from various databases.
8. Run Pig Latin scripts to Sort, Group, Join, Project and Filter the Data

COURSE OUTCOMES:

- CO1:** Describe big data and use cases from selected business domains.
- CO2:** Explain NoSQL big data management.
- CO3:** Install, configure, and run Hadoop and HDFS.
- CO4:** Perform map-reduce analytics using Hadoop.

INDEX

S. No	DATE	NAME OF THE EXPERIMENTS	PAGE No.	MARKS				STAFF SIGNATURE
				Practical Assessment (25)	Record (15)	Viva (10)	Total (50)	
1.		DOWNLOADING AND INSTALLING HADOOP; UNDERSTANDING DIFFERENT HADOOP MODES. STARTUP SCRIPTS, CONFIGURATION FILES.	6					
2.		HADOOP IMPLEMENTATION OF FILE MANAGEMENT TASKS, SUCH AS ADDING FILES AND DIRECTORIES, RETRIEVING FILES AND DELETING FILES	8					
3.		IMPLEMENT OF MATRIX MULTIPLICATION WITH HADOOP MAP REDUCE	10					
4.		RUN A BASIC WORD COUNT MAP REDUCE PROGRAM TO UNDERSTAND MAP REDUCE PARADIGM.	13					
5.		INSTALLATION OF HIVE ALONG WITH PRACTICE EXAMPLES.	16					
6.		INSTALLATION OF HBASE, INSTALLING THRIFT ALONG WITH PRACTICE EXAMPLES	18					
7.		PRACTICE IMPORTING AND EXPORTING DATA FROM VARIOUS DATABASES.	20					
8.		RUN PIG LATIN SCRIPTS TO SORT, GROUP, JOIN, PROJECT AND FILTER THE DATA	22					

AIM

Perform setting up and Installing Hadoop in its three operating modes:

- Standalone
- Pseudo Distributed
- Fully Distributed

DESCRIPTION

Hadoop is written in Java, so you will need to have Java installed on your machine, Version 6 or later. Sun's JDK is the one most widely used with Hadoop, although others have been reported to work.

Hadoop runs on Unix and on Windows. Linux is the only supported production platform, but other flavors of Unix (including Mac OS X) can be used to run Hadoop for development. Windows is only supported as a development platform, and additionally requires Cygwin to run. During the Cygwin installation process, you should include the open ssh package if you plan to run Hadoop in pseudo-distributed mode.

ALGORITHM

1. Command for installing ssh is "sudo apt-get install ssh"
2. Command for key generation is `ssh-keygen -t rsa -P ""`.
3. Store the key into `rsa.pub` by using the command `cat $HOME/.ssh/id_rsa.pub >> $HOME/.ssh/authorized_keys`
4. Extract the java by using the command `tar xvfz jdk-8u60-linux-i586.tar.gz`.
5. Extract the eclipse by using the command `tar xvfz eclipse-jee-mars-R-linux-gtk.tar.gz`
6. Extract the hadoop by using the command `tar xvfz hadoop-2.7.1.tar.gz`
7. Move the java to `/usr/lib/jvm/` and eclipse to `/opt/` paths. Configure the java path in the Eclipse.ini file
8. Export java path and hadoop path in `./bashrc`
9. Check the installation successful or not by checking the java version and hadoop version
10. Check the hadoop instance in standalone mode working correctly or not by using an implicit hadoop jar file named as word count.
11. If the word count is displayed correctly in `part-r-00000` file it means that standalone mode is installed successfully.

STEPS INVOLVED IN INSTALLING HADOOP IN PSEUDO DISTRIBUTED MODE

1. In order to install pseudo distributed mode we need to configure the hadoop configuration files which reside in the directory `/home/lendi/hadoop-2.7.1/etc/hadoop`
2. First configure the `hadoop-env.sh` file by changing the java path.
3. Configure the `core-site.xml` which contains a property tag, it contains name and value. Name as `fs.defaultFS` and value as `hdfs://localhost:9000`
4. Configure `hdfs-site.xml`.
5. Configure `yarn-site.xml`.
6. Configure `mapred-site.xml` before configuring the `copy mapred-site.xml.template` to `mapred-site.xml`.
7. Now format the name node by using the command `hdfs namenode -format`.

8. Type the command `start-dfs.sh, start-yarn.sh` means that starts the daemons like NameNode, DataNode, SecondaryNameNode, ResourceManager, NodeManager.
9. Run JPS which views all daemons. Create a directory in the hadoop by using command `hdfs dfs -mkdir /csedir` and enter some data into `lendi.txt` using command `nano lendi.txt` and copy from local directory to hadoop using command `hdfs dfs -copyFromLocal lendi.txt /csedir/` and run sample jar file wordcount to check whether pseudo distributed mode is working or not.
10. Display the contents of file by using command `hdfs dfs -cat /newdir/part-r-00000`.

STEPS INVOLVED IN INSTALLING HADOOP IN PSEUDO DISTRIBUTED MODE

1. Stop all single node clusters
 - a. `stop-all.sh`
2. Decide one as NameNode (Master) and remaining as DataNodes (Slaves).
3. Copy public key to all three hosts to get a password less SSH access
 - a. `$ ssh-copy-id -I $HOME/.ssh/id_rsa.pub lendi@l5sys24`
4. Configure all Configuration files, to name Master and Slave Nodes.
 - a. `$ cd $HADOOP_HOME/etc/Hadoop`
 - b. `$ nano core-site.xml`
 - c. `$ nano hdfs-site.xml`
5. Add hostnames to file slaves and save it.
 - a. `$ nano slaves`
6. Configure
 - a. `$ nano yarn-site.xml`
7. Do in Master Node
 - a. `$ hdfs namenode -format`
 - b. `$ start-dfs.sh`
 - c. `$ start-yarn.sh`
8. Format NameNode
9. Daemons Starting in Master and Slave Nodes
10. END

INPUT

ubuntu @localhost> jps

OUTPUT

Data node, name node, Secondary name node, NodeManager, Resource Manager

RESULT

Thus, the experiment concludes with the successful installation of Hadoop in various modes, verified startup scripts, and proper configuration file setup.

AIM

Implement the following file management tasks in Hadoop:

- Adding files and directories
- Retrieving files
- Deleting Files

DESCRIPTION

HDFS is a scalable distributed file system designed to scale to petabytes of data while running on top of the underlying file system of the operating system. HDFS keeps track of where the data resides in a network by associating the name of its rack (or network switch) with the dataset. This allows Hadoop to efficiently schedule tasks to those nodes that contain data, or which are nearest to it, optimizing bandwidth utilization. Hadoop provides a set of command line utilities that work similarly to the Linux file commands, and serve as your primary interface with HDFS. We 're going to have a look into HDFS by interacting with it from the command line. We will take a look at the most common file management tasks in Hadoop, which include:

- Adding files and directories to HDFS
- Retrieving files from HDFS to local file system
- Deleting files from HDFS

ALGORITHM**SYNTAX AND COMMANDS TO ADD, RETRIEVE AND DELETE DATA FROM HDFS****STEP-1: ADDING FILES AND DIRECTORIES TO HDFS**

Before you can run Hadoop programs on data stored in HDFS, you'll need to put the data into HDFS first. Let's create a directory and put a file in it. HDFS has a default working directory of /user/\$USER, where \$USER is your login user name. This directory isn't automatically created For you, though, so let's create it with the mkdir command. For the purpose of illustration, we Use chuck. You should substitute your user name in the example commands.

- Hadoop fs -mkdir /user/chuck
- Hadoop fs -put example.txt
- Hadoop fs -put example.txt /user/chuck

STEP-2: RETRIEVING FILES FROM HDFS

The Hadoop command get copies files from HDFS back to the local file system. To retrieve example.txt, we can run the following command:

```
hadoop fs -cat example.txt
```

STEP-3: DELETING FILES FROM HDFS

Hadoop fs -rm example.txt

- Command for creating a directory in hdfs is "hdfs dfs -mkdir /lendicse".
- Adding directory is done through the command "hdfs dfs -put lendi_english /"

STEP-4: COPYING DATA FROM NFS TO HDFS

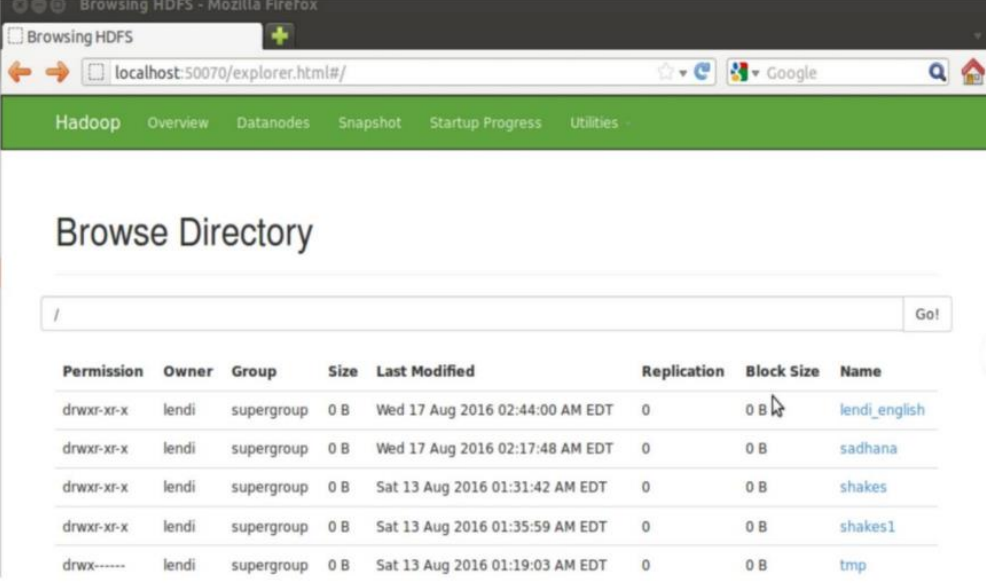
Copying from directory command is “hdfs dfs –copyFromLocal /home/lendi/Desktop/shakes/glossary /lendicse/”

- View the file by using the command “hdfs dfs –cat /lendi_english/glossary”
- Command for listing of items in Hadoop is “hdfs dfs –ls hdfs://localhost:9000/”.
- Command for Deleting files is “hdfs dfs –rm r /kartheek”.

INPUT

Input as any data format of type structured, Unstructured or Semi Structured

OUTPUT



Permission	Owner	Group	Size	Last Modified	Replication	Block Size	Name
drwxr-xr-x	lendi	supergroup	0 B	Wed 17 Aug 2016 02:44:00 AM EDT	0	0 B	lendi_english
drwxr-xr-x	lendi	supergroup	0 B	Wed 17 Aug 2016 02:17:48 AM EDT	0	0 B	sadhana
drwxr-xr-x	lendi	supergroup	0 B	Sat 13 Aug 2016 01:31:42 AM EDT	0	0 B	shakes
drwxr-xr-x	lendi	supergroup	0 B	Sat 13 Aug 2016 01:35:59 AM EDT	0	0 B	shakes1
drwx-----	lendi	supergroup	0 B	Sat 13 Aug 2016 01:19:03 AM EDT	0	0 B	tmp

RESULT

Thus, the Hadoop implementation for file management tasks, including adding files and directories, retrieving files, and deleting files, has been successfully executed and verified.

AIM

To write a Map Reduce Program that implements Matrix Multiplication

DESCRIPTION

We can represent a matrix as a relation (table) in RDBMS where each cell in the matrix can be represented as a record (i,j,value). As an example let us consider the following matrix and its representation. It is important to understand that this relation is a very inefficient relation If the matrix is dense. Let us say we have 5 Rows and 6 Columns , then we need to store only 30 values. But if you consider above relation we are storing 30 row_id, 30 col_id and 30 values in other sense we are tripling the data. So a natural question arises why we need to store in this Format ? In practice most of the matrices are sparse matrices . In sparse matrices not all cells used to have any values , so we don't have to store those cells in DB. So this turns out to be very Efficient in storing such matrices.

MAP REDUCE LOGIC

Logic is to send the calculation part of each output cell of the result matrix to a reducer. So in matrix multiplication the first cell of output (0,0) has multiplication and summation of elements from row 0 of the matrix A and elements from col 0 of matrix B. To do the computation of value in the output cell (0,0) of resultant matrix in a separate reducer we need to use (0,0) as output key of map phase and value should have array of values from row 0 of matrix A and column 0 of matrix B. Hopefully this picture will explain the point. So in this algorithm output from map phase should be having a , where key represents the output cell location (0,0) , (0,1) etc.. and value will be list of all values required for reducer to do computation. Let us take an example for calculating value at output cell (00). Here we need to collect values from row 0 of matrix A and col 0 of matrix B in the map phase and pass (0,0) as key. So a single reducer can do the calculation.

ALGORITHM

We assume that the input files for A and B are streams of (key,value) pairs in sparse matrix format, where each key is a pair of indices (I,j) and each value is the corresponding matrix element value. The output files for matrix $C=A*B$ are in the same format.

We have the following input parameters:

- The path of the input file or directory for matrix A.
- The path of the input file or directory for matrix B.
- The path of the directory for the output files for matrix C.
- strategy = 1, 2, 3 or 4. • R = the number of reducers.
- I = the number of rows in A and C.
- K = the number of columns in A and rows in B.
- J = the number of columns in B and C.
- IB = the number of rows per A block and C block.
- KB = the number of columns per A block and rows per B block.
- JB = the number of columns per B block and C block.

In the pseudo-code for the individual strategies below, we have intentionally avoided factoring common code for the purposes of clarity.

Note that in all the strategies the memory footprint of both the mappers and the reducers is flat at scale.

Note that the strategies all work reasonably well with both dense and sparse matrices. For sparse matrices we do not emit zero elements. That said, the simple pseudo-code for multiplying the individual blocks shown here is certainly not optimal for sparse matrices. As a learning exercise, Our focus here is on mastering the Map Reduce complexities, not on optimizing the sequential matrix multiplication algorithm for the individual blocks

STEPS

1. setup ()
2. var NIB = (I-1)/IB+1
3. var NKB = (K-1)/KB+1
4. var NJB = (J-1)/JB+1
5. map (key, value)
6. if from matrix A with key=(I,k) and value=a(I,k)
7. for $0 \leq j_b < N_{JB}$
8. emit (i/IB, k/KB, j_b, 0), (I mod IB, k mod KB, a(I,k))
9. if from matrix B with key=(k,j) and value=b(k,j)
10. for $0 \leq i_b < N_{IB}$
emit (i_b, k/KB, j/JB, 1), (k mod KB, j mod JB, b(k,j))
Intermediate keys (i_b, k_b, j_b, m) sort in increasing order first by i_b, then by k_b, then by j_b,
Then by m. Note that m = 0 for A data and m = 1 for B data.
The partitioner maps intermediate key (i_b, k_b, j_b, m) to a reducer r as follows:
11. $r = ((i_b * J_B + j_b) * K_B + k_b) \bmod R$
12. These definitions for the sorting order and partitioner guarantee that each reducer R[i_b,k_b,j_b] receives the data it needs for blocks A[i_b,k_b] and B[k_b,j_b], with the data for the A block immediately preceding the data for the B block.
13. var A = new matrix of dimension IBxKB
14. var B = new matrix of dimension KBxJB
15. var sib = -1
16. var skb = -1
Reduce (key, valueList)
17. If key is (i_b, k_b, j_b, 0)
18. // Save the A block.
19. sib = i_b
20. skb = k_b
21. Zero matrix A
22. for each value = (I, k, v) in valueList A(I,k) = v
23. if key is (i_b, k_b, j_b, 1)
24. if $i_b \neq sib$ or $k_b \neq skb$ return // A[i_b,k_b] must be zero!
25. // Build the B block.
26. Zero matrix B
27. for each value = (k, j, v) in valueList B(k,j) = v
28. // Multiply the blocks and emit the result.
29. ibase = i_b*IB
30. jbase = j_b*JB
31. for $0 \leq I < \text{row dimension of A}$
32. for $0 \leq j < \text{column dimension of B}$
33. sum = 0

34. for $0 \leq k < \text{column dimension of } A = \text{row dimension of } B$
 $\text{sum} += A(I,k)*B(k,j)$
35. if $\text{sum} \neq 0$ emit $(\text{ibase}+I, \text{jbase}+j), \text{sum}$

INPUT

Set of Data sets over different Clusters are taken as Rows and Columns

OUTPUT



The screenshot shows two windows. The top window is a terminal with the command `hadoop jar MatrixMultiplication.jar /matrix_data/ /matrix_output_new` entered. The bottom window is a gedit editor showing the output of the job, which consists of six lines of text: `0,0,240.0`, `0,1,250.0`, `0,2,260.0`, `1,0,880.0`, `1,1,930.0`, and `1,2,980.0`.

RESULT

Thus, the matrix multiplication implementation using Hadoop Map Reduce has been successfully completed and verified.

AIM

To Run a basic Word Count Map Reduce Program to understand Map Reduce Paradigm.

DESCRIPTION

Map Reduce is the heart of Hadoop. It is this programming paradigm that allows for massive scalability across hundreds or thousands of servers in a Hadoop cluster. The Map Reduce concept is fairly simple to understand for those who are familiar with clustered scale-out data processing solutions. The term Map Reduce actually refers to two separate and distinct tasks that Hadoop programs perform. The first is the map job, which takes a set of data and converts it into another set of data, where individual elements are broken down into tuples (key/value pairs). The reduce job takes the output from a map as input and combines those data tuples into a smaller set of tuples. As the sequence of the name Map Reduce implies, the reduce job is always performed after the map job.

ALGORITHM**MAP REDUCE PROGRAM**

Word Count is a simple program which counts the number of occurrences of each word in a given text input data set. Word Count fits very well with the Map Reduce programming model making it a great example to understand the Hadoop Map/Reduce programming style. Our implementation consists of three main parts:

1. Mapper
2. Reducer
3. Driver

STEP-1: WRITE A MAPPER

A Mapper overrides the `map()` function from the Class "org.apache.hadoop.mapreduce.Mapper" which provides pairs as the input. A Mapper implementation may output pairs using the provided Context Input value of the WordCount Map task will be a line of text from the input data file and the key would be the line number . Map task outputs for each word in the line of text.

PSEUDO-CODE

```
void Map (key, value)
{
  for each word x in value:
    output.collect(x, 1);
}
```

STEP-2: WRITE A REDUCER

A Reducer collects the intermediate `<key,value>` output from multiple map tasks and assemble a single result. Here, the WordCount program will sum up the occurrence of each word to pairs as `<word, occurrence>`

PSEUDO-CODE

```
void Reduce (keyword,<list of value>)  
{  
  for each x in <list of value>:  
    sum+=x;  
    final_output.collect(keyword, sum);  
}
```

STEP-2: WRITE DRIVER

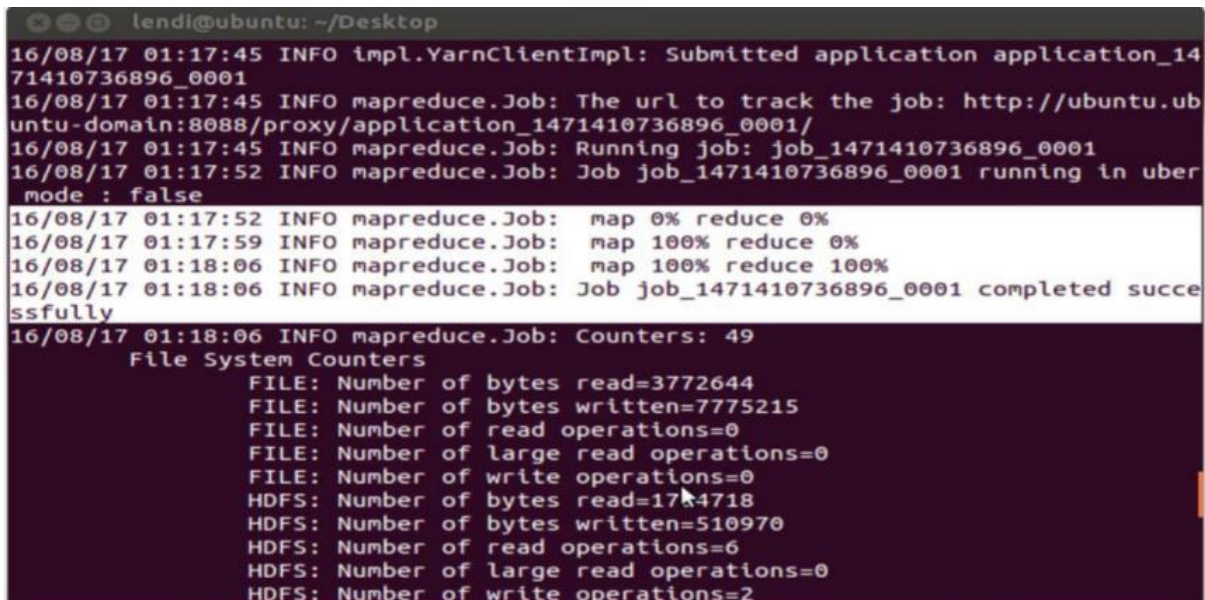
The Driver program configures and run the Map Reduce job. We use the main program to perform basic configurations such as:

- Job Name: name of this Job
- Executable (Jar) Class: the main executable class. For here, Word Count.
- Mapper Class: class which overrides the "map" function. For here, Map.
- Reducer: class which override the "reduce" function. For here , Reduce.
- Output Key: type of output key. For here, Text.
- Output Value: type of output value. For here, IntWritable.
- File Input Path
- File Output Path

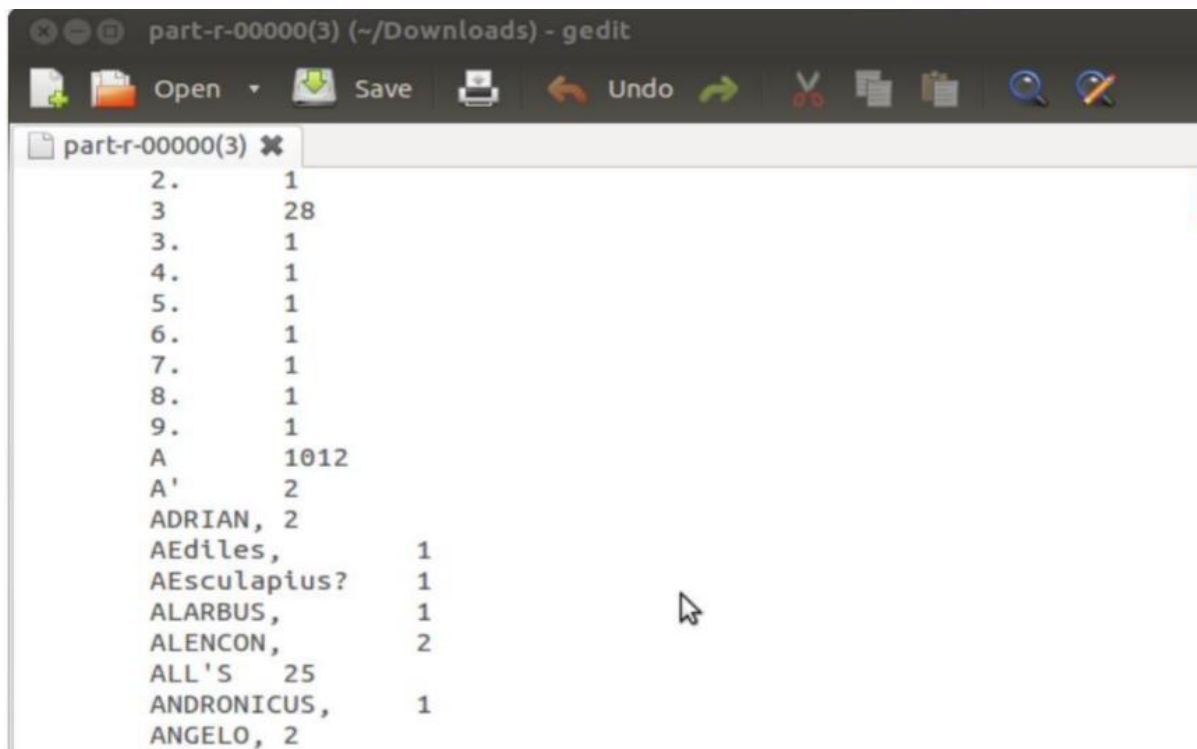
INPUT

Set of Data Related Shakespeare Comedies, Glossary, Poems

OUTPUT



```
lendi@ubuntu: ~/Desktop  
16/08/17 01:17:45 INFO impl.YarnClientImpl: Submitted application application_1471410736896_0001  
16/08/17 01:17:45 INFO mapreduce.Job: The url to track the job: http://ubuntu.ubuntu-domain:8088/proxy/application_1471410736896_0001/  
16/08/17 01:17:45 INFO mapreduce.Job: Running job: job_1471410736896_0001  
16/08/17 01:17:52 INFO mapreduce.Job: Job job_1471410736896_0001 running in uber mode : false  
16/08/17 01:17:52 INFO mapreduce.Job:  map 0% reduce 0%  
16/08/17 01:17:59 INFO mapreduce.Job:  map 100% reduce 0%  
16/08/17 01:18:06 INFO mapreduce.Job:  map 100% reduce 100%  
16/08/17 01:18:06 INFO mapreduce.Job: Job job_1471410736896_0001 completed successfully  
16/08/17 01:18:06 INFO mapreduce.Job: Counters: 49  
  File System Counters  
    FILE: Number of bytes read=3772644  
    FILE: Number of bytes written=7775215  
    FILE: Number of read operations=0  
    FILE: Number of large read operations=0  
    FILE: Number of write operations=0  
    HDFS: Number of bytes read=1714718  
    HDFS: Number of bytes written=510970  
    HDFS: Number of read operations=6  
    HDFS: Number of large read operations=0  
    HDFS: Number of write operations=2
```



The screenshot shows a gedit text editor window titled "part-r-00000(3) (~/Downloads) - gedit". The window contains a list of words and their corresponding counts, formatted as follows:

2.	1
3	28
3.	1
4.	1
5.	1
6.	1
7.	1
8.	1
9.	1
A	1012
A'	2
ADRIAN,	2
AEdiles,	1
AEsculapius?	1
ALARBUS,	1
ALENCON,	2
ALL'S	25
ANDRONICUS,	1
ANGELO,	2

RESULT

Thus, the basic Word Count Map Reduce program was executed and successfully verified, providing a clear understanding of the Map Reduce paradigm.

AIM

To write a python program to implement the installation of Hive on the system along with some practical examples.

ALGORITHM

- Step 1: Install PyHive
- Step 2: Connect the hive Server
- Step 3: Create and connect the Database
- Step 4: Query the Table and Run aggregations
- Step 5: Close the cursor and Connection

PROGRAM

Install pyhive:

```
pip install pyhive
```

Install Hive Server and Start it:

```
hive --service hiveserver2
```

Python Code Examples for Hive:

```
from pyhive import hive
```

```
# Connect to Hive server
```

```
conn = hive.connect(host="localhost", port=10000, username="your_username")
```

```
cursor = conn.cursor()
```

```
# Create a database
```

```
cursor.execute("CREATE DATABASE IF NOT EXISTS mydb")
```

```
# Switch to the database
```

```
cursor.execute("USE mydb")
```

```
# Create a table
```

```
cursor.execute("CREATE TABLE IF NOT EXISTS employee (id INT, name STRING, salary DOUBLE)")
```

```
# Load data into the table
```

```
cursor.execute("LOAD DATA LOCAL INPATH '/path/to/employee_data.csv' INTO TABLE employee")
```

```
# Query the table
```

```
cursor.execute("SELECT * FROM employee WHERE salary > 50000")
```

```
results = cursor.fetchall()
```

```
for row in results:
```

```
    print(row)
```

```
# Create an external table
```

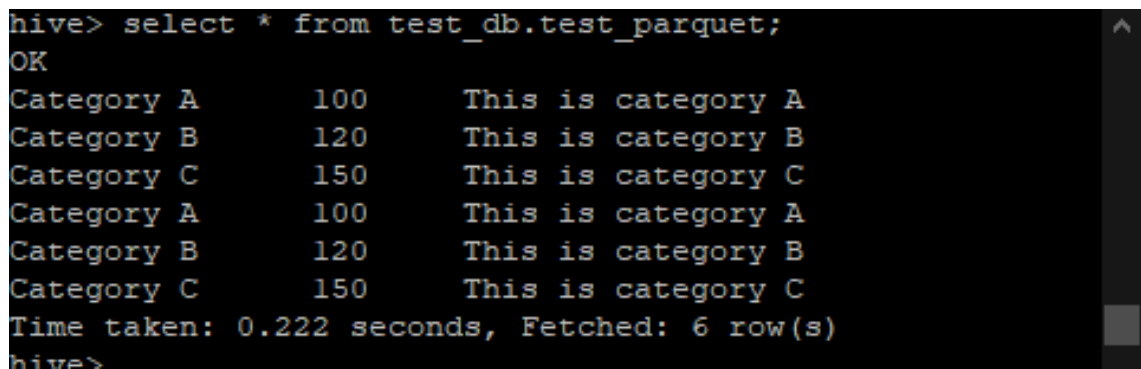


```
cursor.execute("CREATE EXTERNAL TABLE IF NOT EXISTS ext_employee (id INT, name STRING, salary\nDOUBLE) \n\n        LOCATION '/path/to/external_data/'")

# Run aggregations
cursor.execute("SELECT AVG(salary) FROM employee")
result = cursor.fetchone()
print("Average Salary:", result[0])

# Close the cursor and connection
cursor.close()
conn.close()
```

OUTPUT



```
hive> select * from test_db.test_parquet;
OK
Category A      100      This is category A
Category B      120      This is category B
Category C      150      This is category C
Category A      100      This is category A
Category B      120      This is category B
Category C      150      This is category C
Time taken: 0.222 seconds, Fetched: 6 row(s)
hive>
```

RESULT

Thus, the basic Word Count Map Reduce program was executed and successfully verified, providing a clear understanding of the Map Reduce paradigm.

AIM

To write a python program to Installation of HBase, Installing thrift along with Practice examples.

ALGORITHM

- Step 1: Install HBase and thrift
- Step 2: Import the needed package for the practice
- Step 3: Create a connection to HBase and thrift server using protocol
- Step 4: Define and put a data into HBase
- Step 5: Get the data and close the connection of the HBase

PROGRAM

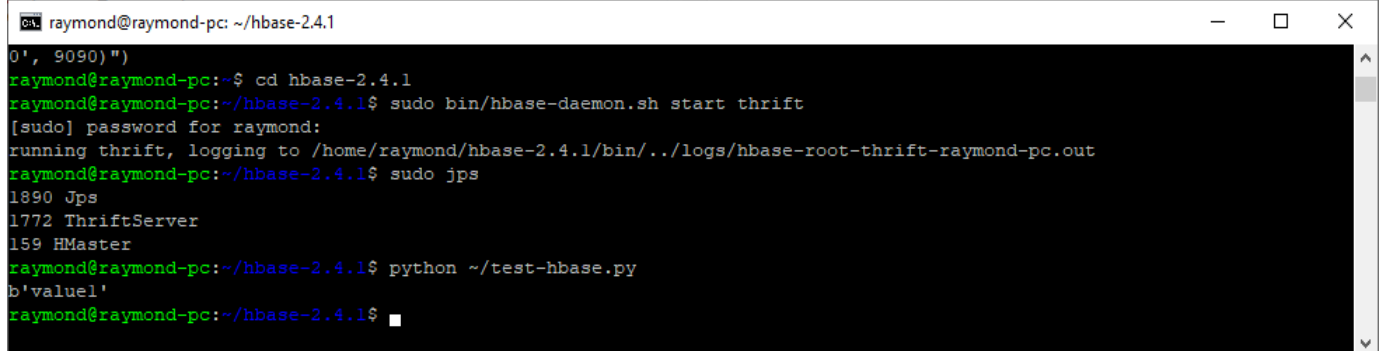
namespace python hbase

```
service HBase {  
    bool put(1: binary row, 2: binary column, 3: binary value),  
    binary get(1: binary row, 2: binary column),  
}  
thrift -r --gen py HBase.thrift
```

Python Code Examples for HBase with Thrift:

```
from thrift.transport import TSocket  
from thrift.transport import TTransport  
from thrift.protocol import TBinaryProtocol  
from hbase import Hbase  
  
# Create a connection to HBase Thrift server  
transport = TSocket.TSocket('localhost', 9090)  
transport = TTransport.TBufferedTransport(transport)  
protocol = TBinaryProtocol.TBinaryProtocol(transport)  
client = Hbase.Client(protocol)  
transport.open()  
  
# Define data for a put operation  
row_key = b'row1'  
column_family = b'cf1'  
qualifier = b'col1'  
value = b'value1'  
  
# Put data into HBase  
client.put(row_key, column_family, qualifier, value, {})  
  
# Get data from HBase  
result = client.get(row_key, column_family, qualifier, {})  
print(result)  
  
# Close the connection  
transport.close()
```

OUTPUT

A terminal window titled 'raymond@raymond-pc: ~/hbase-2.4.1' with standard window controls. The terminal shows the following commands and output:

```
0', 9090)")
raymond@raymond-pc:~$ cd hbase-2.4.1
raymond@raymond-pc:~/hbase-2.4.1$ sudo bin/hbase-daemon.sh start thrift
[sudo] password for raymond:
running thrift, logging to /home/raymond/hbase-2.4.1/bin/../logs/hbase-root-thrift-raymond-pc.out
raymond@raymond-pc:~/hbase-2.4.1$ sudo jps
1890 Jps
1772 ThriftServer
159 HMaster
raymond@raymond-pc:~/hbase-2.4.1$ python ~/test-hbase.py
b'value1'
raymond@raymond-pc:~/hbase-2.4.1$
```

RESULT

Thus, the program to write a python program to Installation of HBase, Installing thrift along with Practice examples.

AIM

To write a python program to implement patrice importing and exporting data from various data bases.

ALGORITHM

- Step 1: Install required libraries such as pyspark and hdfs
- Step 2: Import the necessary package
- Step 3: Create a connection to the pyspark database using python package
- Step 4: Import and export the data using pyspark
- Step 5: Close the Connection

PROGRAM

```
# importing necessary libraries
from pyspark.sql import SparkSession
from pyspark.sql.functions import col

# function to create new SparkSession
def create_session():
    spk = SparkSession.builder \
        .appName("Corona_cases_statewise.com") \
        .getOrCreate()
    return spk

# function to create RDD
def create_RDD(sc_obj,data):
    df = sc.parallelize(data)
    return df

if __name__ == "__main__":

    input_data = [("Uttar Pradesh",122000,89600,12238),
                  ("Maharashtra",454000,380000,67985),
                  ("Tamil Nadu",115000,102000,13933),
                  ("Karnataka",147000,111000,15306),
                  ("Kerala",153000,124000,5259)]

    # calling function to create SparkSession
    spark = create_session()

    # creating spark context object
    sc = spark.sparkContext

    # calling function to create RDD
    rd_df = create_RDD(sc,input_data)

    schema_lst = ["State","Cases","Recovered","Deaths"]
```

```

# creating the dataframe using createDataFrame function
df = spark.createDataFrame(rd_df,schema_lst)

# showing the dataframe and schema
df.printSchema()
df.show()

print("Retrieved Data is:-")

# Retrieving multiple rows using collect() and for loop
for row in df.collect()[0:3]:
    print((row["State"],",",str(row["Cases"]),",", str(row["Recovered"]),",",str(row["Deaths"])))

```

OUTPUT

```

root
 |-- State: string (nullable = true)
 |-- Cases: long (nullable = true)
 |-- Recovered: long (nullable = true)
 |-- Deaths: long (nullable = true)

+-----+-----+-----+-----+
|      State| Cases|Recovered|Deaths|
+-----+-----+-----+-----+
|Uttar Pradesh|122000|   89600| 12238|
|Maharashtra|454000|  380000| 67985|
|Tamil Nadu|115000|   102000| 13933|
|Karnataka|147000|   111000| 15306|
|Kerala|153000|   124000|   5259|
+-----+-----+-----+-----+

Retrieved Data is:-
('Uttar Pradesh', 122000, 89600, 12238)

```

RESULT

Thus, the program to Implement patrice importing and exporting data from various databases.

Ex. No: 08

Date:

**RUN APACHE PIG LATIN SCRIPTS TO SORT, GROUP, JOIN,
PROJECT AND FILTER THE DATA**

AIM

To run Apache pig latin scripts to sort, group, join, project and filter the data

ALGORITHM

- Step 1: Extract the pig-0.15.0.tar.gz and move to home directory
- Step 2: Set the environment of PIG in bashrc file.
- Step 3: Pig can run in two modes
- Step 4: Grunt Shell
- Step 5: LOADING Data into Grunt Shell
- Step 6: Describe Data
- Step 7: DUMP Data
- Step 8: FILTER Data
- Step 9: GROUP Data
- Step 10: Iterating Data
- Step 11: Sorting Data
- Step 12: LIMIT Data
- Step 13: JOIN Data

PROGRAM

Local Mode and Hadoop Mode

Pig -x local and pig

Grunt>

DATA = LOAD <CLASSPATH> USING PigStorage(DELIMITER) as (ATTRIBUTE : DataType1,
ATTRIBUTE : DataType2.....)

Describe DATA;

Dump DATA;

FDATA = FILTER DATA by ATTRIBUTE = VALUE;

GDATA = GROUP DATA by ATTRIBUTE;

FOR_DATA = FOREACH DATA GENERATE GROUP AS GROUP_FUN

ATTRIBUTE = <VALUE>

SORT_DATA = ORDER DATA BY ATTRIBUTE WITH CONDITION;

LIMIT_DATA = LIMIT DATA COUNT;

JOIN DATA1 BY (ATTRIBUTE1,ATTRIBUTE2....) , DATA2 BY (ATTRIBUTE3,ATTRIBUTE....N)

INPUT

Input as Website Click Count Data

OUTPUT

```
lendi@ubuntu: ~  
grunt> ad1 = load '/home/lendi/Desktop/static_data/ad_data/ad_data1.txt' using PigStorage('\t') as (item:chararray,campaignId:chararray,date:chararray,time:chararray,display_site:chararray,was_clicked:int,cpc:int,country:chararray,placement:chararray);  
2016-10-14 02:35:32,441 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum  
2016-10-14 02:35:32,441 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS  
grunt> describe ad1;  
ad1: {item: chararray,campaignId: chararray,date: chararray,time: chararray,display_site: chararray,was_clicked: int,cpc: int,country: chararray,placement: chararray}  
grunt> ad2 = load '/home/lendi/Desktop/static_data/ad_data/ad_data2.txt' using PigStorage(',') as (campaignId:chararray,date:chararray,time:chararray,display_site:chararray,placement:chararray,was_clicked:int,cpc:int,item:chararray);  
2016-10-14 02:36:08,732 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - io.bytes.per.checksum is deprecated. Instead, use dfs.bytes-per-checksum  
2016-10-14 02:36:08,732 [main] INFO org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS  
grunt> describe ad2;  
ad2: {campaignId: chararray,date: chararray,time: chararray,display_site: chararray,placement: chararray,was_clicked: int,cpc: int,item: chararray}  
grunt>
```

```
lendi@ubuntu: ~  
grunt> join_data = join ad1 by (campaignId,display_site,cpc),ad2 by (campaignId,display_site,cpc);  
grunt> describe join_data;  
join_data: {ad1::item: chararray,ad1::campaignId: chararray,ad1::date: chararray,ad1::time: chararray,ad1::display_site: chararray,ad1::was_clicked: int,ad1::cpc: int,ad1::country: chararray,ad1::placement: chararray,ad2::campaignId: chararray,ad2::date: chararray,ad2::time: chararray,ad2::display_site: chararray,ad2::placement: chararray,ad2::was_clicked: int,ad2::cpc: int,ad2::item: chararray}  
grunt>
```

RESULT

Thus, running Apache pig latin scripts to sort, group, join, project and filter the data was executed and verified successfully.